Networks Lab-PCCST507
Assignment
Socket Programming

SUBMITTED BY:

ARUNIMA RAJAN

S5 CSE A

ROLL NO:16

VJCET

Question 1 – Hello Client-Server (Beginner)

server.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   char buffer[1024];
   /* Initialize Winsock */
   if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
   {
      printf("WSAStartup Failed\n");
      return 1;
   }
   /* Create Socket */
   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
  if (serverSocket == INVALID_SOCKET)
   {
      printf("Socket Creation Failed\n");
      WSACleanup();
      return 1;
   }
 printf("Socket Created Successfully...\n");
 /* Server Address */
```

```
 serverAddr.sin_family = AF_INET;
 serverAddr.sin_addr.s_addr = INADDR_ANY;
 serverAddr.sin_port = htons(PORT);
/* Bind */
 if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
 {
    printf("Bind Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
 }
printf("Bind Successful...\n");
/* Listen */
 if (listen(serverSocket, 5) == SOCKET_ERROR)
 {
    printf("Listen Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
 }
  printf("Waiting for Client...\n");
  /* Accept Client */
 clientSocket = accept(serverSocket, NULL, NULL);
 if (clientSocket == INVALID_SOCKET)
 {
    printf("Accept Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
 }
```

```
    printf("Client Connected...\n");
   /* Receive Message */
    memset(buffer, 0, sizeof(buffer));
    recv(clientSocket, buffer, sizeof(buffer) - 1, 0);
   printf("Client Says : %s\n", buffer);
   /* Send Reply */
    strcpy(buffer, "Hello Client");
    send(clientSocket, buffer, strlen(buffer) + 1, 0);
    printf("Reply Sent...\n");
   /* Close Connection */
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    printf("Connection Closed...\n");
     return 0;
}
```

client.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
int main()
{
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in serverAddr;
    char buffer[1024];
     /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
```

```
{
    printf("WSAStartup Failed\n");
    return 1;
}
/* Create Socket */
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd == INVALID_SOCKET)
{
    printf("Socket Creation Failed\n");
    WSACleanup();
    return 1;
}
printf("Socket Created Successfully...\n");
/* Server Address */
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
/* Connect to Server */
if (connect(sockfd,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Connection Failed\n");
    closesocket(sockfd);
    WSACleanup();
    return 1;
}
printf("Connected to Server...\n");
/* Enter Message */
printf("Enter Message : ");
fgets(buffer, sizeof(buffer), stdin);
/* Remove newline */
```

```
    buffer[strcspn(buffer, "\n")] = '\0';
    /* Send Message */
    send(sockfd, buffer, strlen(buffer) + 1, 0);
    printf("Message Sent...\n");
    /* Receive Reply */
    memset(buffer, 0, sizeof(buffer));
    recv(sockfd, buffer, sizeof(buffer) - 1, 0);
    printf("\nReply from Server : %s\n", buffer);
    /* Close Connection */
    closesocket(sockfd);
    WSACleanup();
    printf("Connection Closed...\n");
    return 0;
}
```

```
PS C:\Users\aruni\Downloads\Program> gcc server.c -o server.exe -lws2_32
PS C:\Users\aruni\Downloads\Program> .\server.exe
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Client Says : Hello server
Reply Sent...
Connection Closed...
PS C:\Users\aruni\Downloads\Program>
```

```
PS C:\Users\aruni\Downloads\Program> gcc client.c -o client.exe -lws2_32
PS C:\Users\aruni\Downloads\Program> .\client.exe
Socket Created Successfully...
Connected to Server...
Enter Message : Hello server
Message Sent...

Reply from Server : Hello Client
Connection Closed...
PS C:\Users\aruni\Downloads\Program>
```

Question 2 – Arithmetic Server

Server.c:

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
int main()
{
   WSADATA wsa;
   SOCKET serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   int num1, num2;
   double addition, subtraction, multiplication, division;
   /* Initialize Winsock */
   if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
   {
      printf("WSAStartup Failed\n");
      return 1;
   }
   /* Create Socket */
   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if (serverSocket == INVALID_SOCKET)
   {
      printf("Socket Creation Failed\n");
      WSACleanup();
      return 1;
   }
   printf("Socket Created Successfully...\n");
   /* Server Address */
   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);
```

```
/* Bind */
if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Bind Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
printf("Bind Successful...\n");
/* Listen */
if (listen(serverSocket, 5) == SOCKET_ERROR)
{
    printf("Listen Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
printf("Waiting for Client...\n");
/* Accept */
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
    printf("Accept Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
printf("Client Connected...\n");
/* Receive first number */
recv(clientSocket, (char *)&num1, sizeof(num1), 0);
```

```
    /* Receive second number */
    recv(clientSocket, (char *)&num2, sizeof(num2), 0);
    printf("Received Numbers: %d and %d\n", num1, num2);
    /* Perform calculations */
    addition = num1 + num2;
    subtraction = num1 - num2;
    multiplication = num1 * num2;
    if (num2 != 0)
        division = (double)num1 / num2;
    else
        division = 0;
    /* Send results */
    send(clientSocket, (char *)&addition, sizeof(addition), 0);
    send(clientSocket, (char *)&subtraction, sizeof(subtraction), 0);
    send(clientSocket, (char *)&multiplication, sizeof(multiplication), 0);
    send(clientSocket, (char *)&division, sizeof(division), 0);
    printf("Results Sent to Client...\n");
    /* Close */
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    printf("Connection Closed...\n");
    return 0;
}
```

Client.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
```

```
int main()
{
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in serverAddr;
    int num1, num2;
    double addition, subtraction, multiplication, division;
    /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }
    /* Create Socket */
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }
    printf("Socket Created Successfully...\n");
    /* Server Address */
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
    /* Connect */
    if (connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Connection Failed\n");
```

```
    closesocket(sockfd);
    WSACleanup();
    return 1;
}
printf("Connected to Server...\n");
/* Input */
printf("Enter first integer: ");
scanf("%d", &num1);
printf("Enter second integer: ");
scanf("%d", &num2);
/* Send numbers */
send(sockfd, (char *)&num1, sizeof(num1), 0);
send(sockfd, (char *)&num2, sizeof(num2), 0);
printf("\nNumbers Sent to Server...\n");
/* Receive results */
recv(sockfd, (char *)&addition, sizeof(addition), 0);
recv(sockfd, (char *)&subtraction, sizeof(subtraction), 0);
recv(sockfd, (char *)&multiplication, sizeof(multiplication), 0);
recv(sockfd, (char *)&division, sizeof(division), 0);
/* Display results */
printf("\n----- Results -----\n");
printf("Addition       = %.2f\n", addition);
printf("Subtraction    = %.2f\n", subtraction);
printf("Multiplication = %.2f\n", multiplication);
if (num2 != 0)
    printf("Division       = %.2f\n", division);
else
    printf("Division       = Cannot divide by zero\n");
/* Close */
closesocket(sockfd);
WSACleanup();
printf("\nConnection Closed...\n");
```

```
    return 0;
}
```

```
PS C:\Users\aruni\Downloads\Q2> gcc server.c -o server.exe -lws2_32
PS C:\Users\aruni\Downloads\Q2> .\server.exe
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Numbers: 20 and 5
Results Sent to Client...
Connection Closed...
PS C:\Users\aruni\Downloads\Q2>
```

```
PS C:\Users\aruni\Downloads\Q2> gcc client.c -o client.exe -lws2_32
PS C:\Users\aruni\Downloads\Q2> .\client.exe
Socket Created Successfully...
Connected to Server...
Enter first integer: 20
Enter second integer: 5

Numbers Sent to Server...

----- Results -----
Addition       = 25.00
Subtraction    = 15.00
Multiplication = 100.00
Division       = 4.00

Connection Closed...
PS C:\Users\aruni\Downloads\Q2>
```

Question 3 – String Processing Server

Server.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <ctype.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
struct Result
```

```
{
    int length;
    char uppercase[1024];
    char reverse[1024];
};
int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];
    struct Result result;
    int i;
    /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }
    /* Create Socket */
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }
    printf("Socket Created Successfully...\n");
    /* Server Address */
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);
```

```
/* Bind */
if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
   printf("Bind Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Bind Successful...\n");
/* Listen */
if (listen(serverSocket, 5) == SOCKET_ERROR)
{
   printf("Listen Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Waiting for Client...\n");
/* Accept */
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
   printf("Accept Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Client Connected...\n");
/* Receive String */
memset(buffer, 0, sizeof(buffer));
```

```
    recv(clientSocket, buffer, sizeof(buffer) - 1, 0);
    printf("Received String : %s\n", buffer);
    /* Find Length */
    result.length = strlen(buffer);
    /* Convert to Uppercase */
    strcpy(result.uppercase, buffer);
    for (i = 0; result.uppercase[i] != '\0'; i++)
    {
        result.uppercase[i] =
            toupper((unsigned char)result.uppercase[i]);
    }
    /* Reverse String */
    for (i = 0; i < result.length; i++)
    {
        result.reverse[i] =
            buffer[result.length - 1 - i];
    }
    result.reverse[result.length] = '\0';
    /* Send Complete Result */
    send(clientSocket,
         (char *)&result,
         sizeof(result),
         0);
    printf("Results Sent to Client...\n");
    /* Close */
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    printf("Connection Closed...\n");
    return 0;
}
```

```
PS C:\Users\aruni\Downloads\Q3> gcc server.c -o server.exe -lws2_32
PS C:\Users\aruni\Downloads\Q3> .\server.exe
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String : computer
Results Sent to Client...
Connection Closed...
PS C:\Users\aruni\Downloads\Q3>
```

```
PS C:\Users\aruni\Downloads\Q3> gcc client.c -o client.exe -lws2_32
PS C:\Users\aruni\Downloads\Q3> .\client.exe
Socket Created Successfully...
Connected to Server...
Enter a string: computer
String Sent to Server...

----- Results -----
String Length = 8
Uppercase     = COMPUTER
Reverse       = retupmoc

Connection Closed...
PS C:\Users\aruni\Downloads\Q3>
```

Question 4 – Array Processing Server

Server.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
#define MAX 100
struct Result
{
    int sum;
```

```
    float average;
    int maximum;
    int minimum;
};
int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    int n;
    int arr[MAX];
    struct Result result;
    /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }
    /* Create Socket */
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }
    printf("Socket Created Successfully...\n");
    /* Server Address */
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);
    /* Bind */
```

```
  if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
   printf("Bind Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Bind Successful...\n");
/* Listen */
if (listen(serverSocket, 5) == SOCKET_ERROR)
{
   printf("Listen Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Waiting for Client...\n");
/* Accept */
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
   printf("Accept Failed\n");
   closesocket(serverSocket);
   WSACleanup();
   return 1;
}
printf("Client Connected...\n");
/* Receive N */
recv(clientSocket, (char *)&n, sizeof(n), 0);
/* Receive Array */
```

```
recv(clientSocket,
     (char *)arr,
     sizeof(int) * n,
     0);
printf("Received Array: ");
for (int i = 0; i < n; i++)
{
   printf("%d ", arr[i]);
}
printf("\n");
/* Calculate Sum */
result.sum = 0;
for (int i = 0; i < n; i++)
{
   result.sum += arr[i];
}
/* Calculate Average */
result.average = (float)result.sum / n;
/* Find Maximum and Minimum */
result.maximum = arr[0];
result.minimum = arr[0];
for (int i = 1; i < n; i++)
{
   if (arr[i] > result.maximum)
   {
      result.maximum = arr[i];
   }
   if (arr[i] < result.minimum)
   {
      result.minimum = arr[i];
   }
}
```

```
    /* Send Result */
    send(clientSocket,
        (char *)&result,
        sizeof(result),
        0);
    printf("Results Sent to Client...\n");
    /* Close */
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    printf("Connection Closed...\n");
    return 0;
}
```

```
PS C:\Users\aruni\Downloads\Q4> gcc server.c -o server.exe -lws2_32
PS C:\Users\aruni\Downloads\Q4> .\server.exe
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Array: 10 20 5 40 15
Results Sent to Client...
Connection Closed...
PS C:\Users\aruni\Downloads\Q4>
```

```
PS C:\Users\aruni\Downloads\Q4> gcc client.c -o client.exe -lws2_32
PS C:\Users\aruni\Downloads\Q4> .\client.exe
Socket Created Successfully...
Connected to Server...
Enter number of elements: 5
Enter 5 integers:
10
20
5
40
15

Array Sent to Server...

----- Results -----
Sum     = 90
Average = 18.00
Maximum = 40
Minimum = 5

Connection Closed...
PS C:\Users\aruni\Downloads\Q4>
```

Question 5 – Matrix Analyzer

Server.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
#define PORT 8080
#define MAX 10
int receiveInt(SOCKET socket, int *value)
{
   int total = 0;
   int bytes;
   while (total < sizeof(int))
   {
      bytes = recv(socket,
                  ((char *)value) + total,
                  sizeof(int) - total,
                  0);
      if (bytes <= 0)
         return 0;
      total += bytes;
   }
   return 1;
}
int sendInt(SOCKET socket, int value)
{
   int total = 0;
   int bytes;
   while (total < sizeof(int))
   {
      bytes = send(socket,
```

```
                ((char *)&value) + total,
                 sizeof(int) - total,
                 0);
        if (bytes <= 0)
            return 0;
        total += bytes;
    }
    return 1;
}
int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    int n;
    int matrix[MAX][MAX];
    int matrixType;
    int sum;
    int trace;
    int isDiagonal;
    int isUpper;
    int isLower;
    /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }
    /* Create Socket */
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSocket == INVALID_SOCKET)
    {
```

```
    printf("Socket Creation Failed\n");
    WSACleanup();
    return 1;
}
printf("Socket Created Successfully...\n");
/* Server Address */
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
/* Bind */
if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Bind Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
printf("Bind Successful...\n");
/* Listen */
listen(serverSocket, 5);
printf("Waiting for Client...\n");
/* Accept */
clientSocket = accept(serverSocket, NULL, NULL);
if (clientSocket == INVALID_SOCKET)
{
    printf("Accept Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
```

```
printf("Client Connected...\n");
/* Receive matrix size */
if (!receiveInt(clientSocket, &n))
{
   printf("Failed to receive matrix size.\n");
   return 1;
}
printf("Matrix Order = %d\n", n);
if (n <= 0 || n > MAX)
{
   printf("Invalid matrix size.\n");
   return 1;
}
/* Receive every matrix element */
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      if (!receiveInt(clientSocket, &matrix[i][j]))
      {
         printf("Failed to receive matrix element.\n");
         return 1;
      }
   }
}
/* Display received matrix */
printf("\nReceived Matrix:\n");
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      printf("%d ", matrix[i][j]);
```

```
}
    printf("\n");
 }
 /* Calculate Sum */
 sum = 0;
 for (int i = 0; i < n; i++)
 {
    for (int j = 0; j < n; j++)
    {
       sum += matrix[i][j];
    }
 }
 /* Calculate Trace */
 trace = 0;
 for (int i = 0; i < n; i++)
 {
    trace += matrix[i][i];
 }
 /* Check matrix types */
 isDiagonal = 1;
 isUpper = 1;
 isLower = 1;
 for (int i = 0; i < n; i++)
 {
    for (int j = 0; j < n; j++)
    {
       if (i != j && matrix[i][j] != 0)
       {
          isDiagonal = 0;
       }
       if (i > j && matrix[i][j] != 0)
       {
```

```
            isUpper = 0;
        }
        if (i < j && matrix[i][j] != 0)
        {
            isLower = 0;
        }
    }
}
if (isDiagonal)
{
    matrixType = 1;
}
else if (isUpper)
{
    matrixType = 2;
}
else if (isLower)
{
    matrixType = 3;
}
else
{
    matrixType = 4;
}
/* Send results individually */
sendInt(clientSocket, matrixType);
sendInt(clientSocket, sum);
sendInt(clientSocket, trace);
printf("\nResults Sent to Client...\n");
/* Close */
closesocket(clientSocket);
closesocket(serverSocket);
```

```
    WSACleanup();
    printf("Connection Closed...\n");
    return 0;
}
```

```
PS C:\Users\aruni\Downloads\Q5> gcc server.c -o server.exe -lws2_32
PS C:\Users\aruni\Downloads\Q5> .\server.exe
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Matrix Order = 3

Received Matrix:
1 0 0
0 2 0
0 0 3

Results Sent to Client...
Connection Closed...
PS C:\Users\aruni\Downloads\Q5>
```

```
PS C:\Users\aruni\Downloads\Q5> gcc client.c -o client.exe -lws2_32
PS C:\Users\aruni\Downloads\Q5> .\client.exe
Socket Created Successfully...
Connected to Server...
Enter order of matrix: 3
Enter 3 x 3 matrix:
1 0 0
0 2 0
0 0 3

Matrix Sent to Server...

----- Results -----
Matrix Type = Diagonal Matrix
Sum of Matrix Elements = 6
Trace of Matrix        = 6

Connection Closed...
PS C:\Users\aruni\Downloads\Q5>
```